

PROGRAMMAZIONE II

EREDITARIETÀ E POLIMORFISMO

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

EREDITARIETÀ



NON EREDITARIETÀ



SOTTOTIPI

Un tipo può essere visto come l'insieme di tutti i valori con qualcosa in comune

Prime: interi ≥ 1 con solo due divisori $\subseteq$ **Nat:** interi ≥ 0

Se tutti i valori di un tipo appartengono a un altro tipo, il primo è un **sottotipo** del secondo

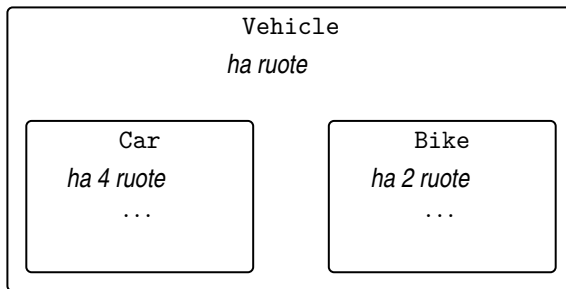
Le classi sono tipi, poiché rappresentano insiemi di oggetti con dati/operazioni comuni

- Una classe può essere un sottotipo, o **sottoclasse**, di un'altra classe

SOTTOCLASSE

La classe `Car` è una sottoclasse della classe `Vehicle`

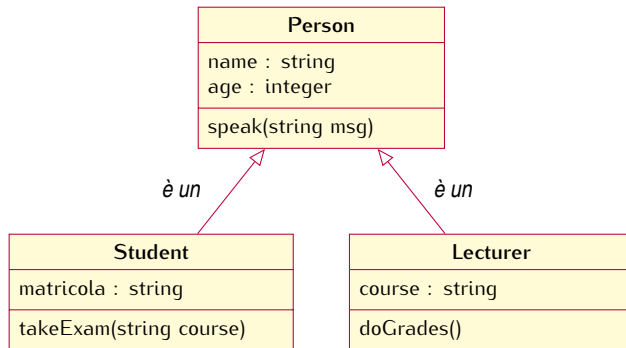
- ▶ Anche la classe `Bike` è una sottoclasse di `Vehicle`
- ▶ Ma non è una sottoclasse di `Car`



SOTTOCLASSE (CONT.)

Ogni studente è una persona, ma non un docente

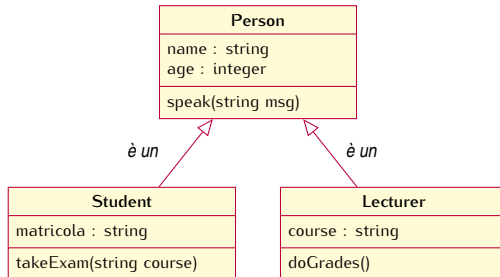
- Gli oggetti `Student` e `Lecturer` hanno tutto ciò che ha un oggetto `Person`



`Student` è una **sottoclasse** di `Person`

`Person` è una **superclasse** di `Student`

EREDITARIETÀ



Una sottoclasse **eredita** tutti i campi e metodi della sua superclasse

- Una sottoclasse si dice derivata dalla sua superclasse

EREDITARIETÀ (CONT.)

Una sottoclasse può

// le modifiche sono disponibili solo nella sottoclasse

- ▶ Ridefinire, o effettuare l'**override**, di alcuni metodi ereditati dalla superclasse
- ▶ Aggiungere nuovi campi e metodi

```
public class Student extends Person { ... }
```

L'ereditarietà in Java offre un solo tipo di derivazione

- ▶ La visibilità di classi, campi e metodi non può deviare dalla superclasse

SUPERCLASSE

```
public class Vector {                                     1
    private int[] vect;                                   2
    public void putElement(int e) { ... // aggiunge l'elemento 3
}                                                         4
                                                         5
public class OrderedVector extends Vector {              6
    private Order orde // campo aggiunto                7
    // metodo sovrascritto                               8
    public void putElement(int e) {                     9
        if (orde == Order.ASC) { ... } // aggiunge e ordina in modo crescente 10
        else { ... // aggiunge e ordina in modo decrescente 11
    }                                                    12
    public Order getOrder() { return orde; // metodo aggiunto 13
}                                                         14
```

Possiamo riusare il codice di una superclasse?

SUPERCLASSE (CONT.)

Con la parola chiave **super** possiamo riferirci all'oggetto della classe genitore

- Così da poter chiamare i metodi della classe genitore

```
public class Vector {                                1
    private int[] vect;                               2
    public void putElement(int e) { ... // aggiunge l'elemento 3
}                                                       4

public class OrderedVector extends Vector {          5
    ...                                               6
    public void putElement(int e) {                  7
        super.putElement(e); // l'inserimento in coda è ora qui 8
        if (order == Order.ASC // qui ordina solo in modo crescente 9
            else { ... // qui ordina solo in modo decrescente 10
        }                                           11
    }                                           12
}                                           13
```

PERCHÉ L'EREDITARIETÀ?

Spesso, una classe è solo una modifica di un'altra classe

- ▶ L'ereditarietà riduce al minimo la ripetizione dello stesso codice

Localizzazione del codice

- ▶ Correggere un bug in una classe lo corregge automaticamente nelle sottoclassi
- ▶ Aggiungere una nuova funzionalità a una classe la aggiunge anche nelle sottoclassi
- ▶ Minori probabilità di implementazioni diverse (inconsistenti) della stessa funzione

NOMENCLATURA DELL'EREDITARIETÀ

Classe genitore

- Classe un livello sopra

Classe figlia

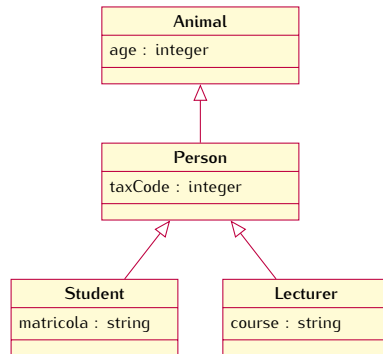
- Classe un livello sotto

Superclasse, o classe antenata, o classe base

- Classe uno o più livelli sopra

Sottoclasse, o classe discendente

- Classe uno o più livelli sotto



! Java non supporta l'ereditarietà multipla

RISOLUZIONE DI METODI E CAMPI

La JVM cerca le definizioni dei metodi nella catena delle superclassi

- ▶ Se un metodo è definito (o ridefinito) nella classe corrente, viene chiamato
- ▶ Se non trovato, il metodo viene cercato nella classe genitore
- ▶ Se non trovato, il metodo viene cercato nel genitore della classe genitore
- ▶ ... *// è garantito che la ricerca prima o poi si fermi*

! Tecnicamente anche i campi possono essere ridefiniti, ma è meglio non farlo

- ▶ Per rendere il codice più comprensibile
- ▶ Per evitare bug (overload accidentale anziché override)

Se un metodo con `@Override` non è nella superclasse, si ottiene un errore di compilazione

```
public class Vector {
    public void putElement(int e) { ... }
}

public class OrderedVector extends Vector {
    @Override
    public void putElment(int e) { ... } // errore di compilazione
}
```

VISIBILITÀ DELLE SOTTOCLASSI

```
public class Person { // // dichiarata nel package A      1
    String name;                                           2
}                                                         3

public class Student extends Person { // // dichiarata nel package B      4
    String matricola;                                       5
    void printInfo() {                                       6
        System.out.println(name + "□" + matricola); // // errore di compilazione      7
                                                    // name non accessibile      9
    }                                                         10
}
```

Quando omissso, l'accesso di default ai membri di una classe è **package-private**

- Non consente l'accesso alle sottoclassi di altri package

VISIBILITÀ DEI MEMBRI DI UNA CLASSE

Il modificatore di accesso `protected` consente l'accesso alle sottoclassi di altri package

Modificatori di accesso alle classi

- ▶ `public`
- ▶ `package-private` (omesso)

Modificatori di accesso ai membri

- ▶ `public`
- ▶ `protected`
- ▶ `package-private` (omesso)
- ▶ `private`

REGOLE DI ACCESSO RIVISITATE

Regole di accesso per i membri di una classe (campi e metodi)

	stesso package		altro package	
	stessa classe	altra classe	sottoclasse	non sottoclasse
membro private in qualsiasi classe	✓	✗	✗	✗
membro (package) in qualsiasi classe	✓	✓	✗	✗
membro protected in classe (package)	✓	✓	✗	✗
membro public in classe (package)	✓	✓	✗	✗
membro protected in classe public	✓	✓	✓	✗
membro public in classe public	✓	✓	✓	✓

GERARCHIA DELLE CLASSI



L'ANELLO UNICO

Tutte le classi in Java sono discendenti della classe `Object`

- ▶ Classi di libreria, classi utente, classi wrapper, enum, array

Il compilatore esegue implicitamente la derivazione

```
class Car { ... }  →  class Car extends Object { ... }
```

La classe `Object` contiene alcuni metodi di utilità ereditati da tutte le classi

- ▶ `public String toString()` *// restituisce il riferimento come stringa*
- ▶ `public boolean equals(Object obj)` *// confronta i riferimenti*
- ▶ `public int hashCode()` *// restituisce un id univoco per l'oggetto*

L'ANELLO UNICO (CONT.)

Spesso, i metodi ereditati da `Object` non sono soddisfacenti

- Vengono tipicamente ridefiniti nelle sottoclassi

```
class Person {                                     1
    String name = "Sam";                           2

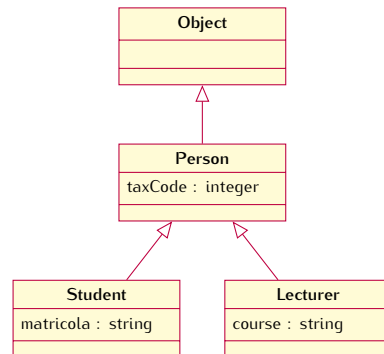
    @Override                                       3
    public String toString() { return name; }        4

    void printPerson() {                            5
        System.out.println(super.toString() // stampa 'Person@5ca881b5' 6
        System.out.println(this.toString()) // stampa 'Sam'             7
    }                                              8
}                                                  9
}                                                  10
}                                                  11
```

CLASSE DI ORIGINE E SUPERCLASSI

In `Student sam = new Student();`

- ▶ La classe di origine dell'oggetto `sam` è `Student`
- ▶ Le superclassi dell'oggetto `sam` sono `Person` e `Object`

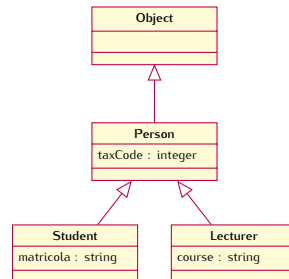


CLASSE DI ORIGINE E SUPERCLASSI (CONT.)

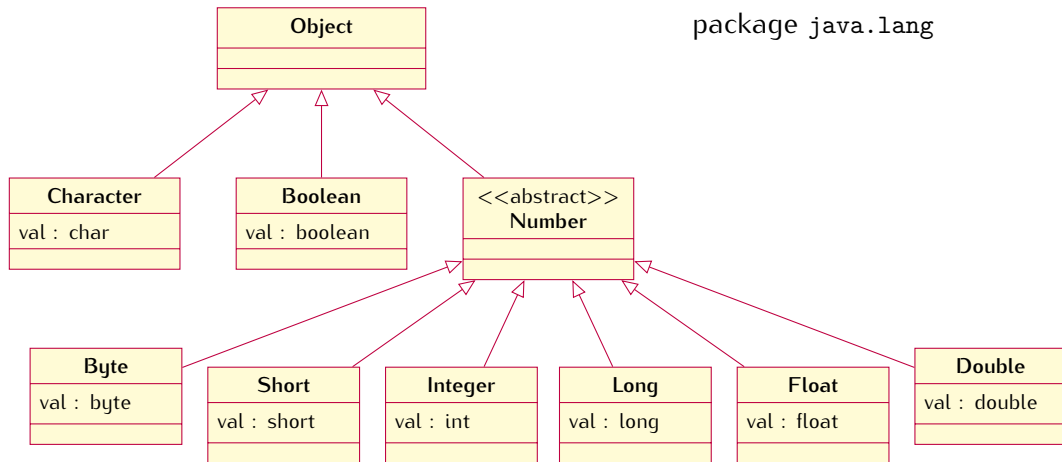
Java permette di verificare se una classe è la classe di origine o superclasse di un oggetto

obj instanceof Clazz

```
Student sam = new Student();           1
                                         2
(sam instanceof Student) // true        3
(sam instanceof Person)  // true        4
(sam instanceof Lecturer) // false      5
(sam instanceof Object)  // sempre true  6
```



GERARCHIA DELLE CLASSI WRAPPER



UGUAGLIANZA VS IDENTITÀ TRA OGGETTI

Il metodo `equals()` dovrebbe verificare l'uguaglianza tra oggetti

- In `Object` verifica invece l'uguaglianza dei riferimenti (identità tra oggetti)

```
class Person {                                1
    String name;                               2
    int age;                                   3
}                                                4

Person sam = new Person("Sam", 21);           5
Person alsoSam = new Person("Sam", 21);       6
System.out.println(sam.equals(alsoSam) // l'output è 'false' 7
                                                8
```

Qui `sam` e `alsoSam` non sono lo stesso oggetto, ma sono uguali

UGUAGLIANZA VS IDENTITÀ TRA OGGETTI (CONT.)

Ridefinire il metodo `equals()` per ottenere il risultato desiderato

- Due oggetti `Person` sono uguali quando hanno gli stessi `name` ed `age`

```
@Override
public boolean equals(Object obj)
    if (obj == this) return true; // // prima controlla l'identità

    if (obj == null) return false;
    if (!(obj instanceof Person)) // poi la compatibilità del tipo
        Person other = (Person) obj; // cast esplicito, si veda dopo

    // infine entra nell'oggetto per confrontare campo per campo

    if (this.name == null)
        if (other != null) return false;
    else
        if (!this.name.equals(other.name)) return false;

    return true; // se tutti i controlli passano, sono uguali
}
```

CONTRATTO DI UGUAGLIANZA

L'implementazione del metodo `equals()` dovrebbe rispettare alcuni vincoli

Assiomi della relazione di equivalenza

Riflessività `x.equals(x) == true`

Simmetria `x.equals(y) == y.equals(x)`

Transitività `x.equals(y) == true == y.equals(z) implica x.equals(x) == true`

Consistenza

- ▶ Esecuzioni multiple di `x.equals(y)` devono restituire lo stesso risultato
- ▶ Nessun oggetto è uguale a `null`, ossia `x.equals(null) == false`

EREDITARIETÀ E COSTRUTTORI

Poiché una sottoclasse estende la classe genitore, servono (almeno) due costruttori

- ▶ Il compilatore chiama implicitamente il costruttore di default della classe genitore
- ▶ Il costruttore della classe genitore viene eseguito prima di quello della classe figlia

L'esecuzione dei costruttori procede dall'alto verso il basso nella gerarchia

- ▶ Quando viene eseguito un metodo della classe figlia (costruttore incluso), la superclasse è già completamente inizializzata

IL SUPERPOTERE DEI COSTRUTTORI

Con `super()` un costruttore può riferirsi ai costruttori del genitore

```
class Person {                                     1
    String name;                                   2
    Person() { }    // costruttore di default aggiunto implicitamente 3
}                                                    4

class Student extends Person {                    5
    int matricola;                                  6
    Student() {    // costruttore di default aggiunto implicitamente 8
        super();    // chiamata nascosta al costruttore genitore 9
    }                                                  10
}                                                    11
```

// Le righe 3, 8, 9 e 10 non compariranno effettivamente nel codice

IL SUPERPOTERE DEI COSTRUTTORI (CONT.)

Se nella classe genitore è definito un costruttore con parametri

- Nessun costruttore di default viene inserito nella classe figlia

```
class Person {                                1
    String name;                               2
    Person(String name) { this.name = name // costruttore con parametri
}                                              4
                                              5

class Student extends Person {                6
    int matricola;                             7
    // non possiamo compilare questa classe  8
}                                              9
```

Aggiungere un costruttore di default `Person() { }` nella classe genitore oppure aggiungere un costruttore nella classe figlia che chiama `super("Sam")` come prima istruzione

OVERLOADING DI SUPER

Dato che possiamo avere molti costruttori, l'overloading permette a `super()` di sceglierne uno

```
class Person {
    String name;
    Person() { }    // costruttore di default
    Person(String name) { this.name = name // costruttore con parametri
}

class Student extends Person {
    int matricola;
    Student(String name, int matricola) {
        super(name); // th // questa deve essere la prima istruzione
        this.matricola = matricola // il costruttore genitore viene selezionato
    }
}
```

1
2
3
5
6
7
8
9
10
13

OSSERVAZIONE FINALE

Si può impedire l'override di un metodo usando la parola chiave `final`

- ▶ Utile quando il metodo non deve cambiare
- ▶ Ad esempio, quando il metodo fornisce un servizio di base ad altri metodi
- ▶ Il tentativo di effettuare l'override di un metodo `final` genererà un errore di compilazione

POLIMORFISMO



STESSO TERMINE, COMPORTAMENTI DIVERSI



POLIMORFISMO DI SOTTOTIPO

Un riferimento di tipo T può puntare a un oggetto di tipo S se e solo se

S è T oppure S è una sottoclasse di T

```
Person perso // la classe Student estende Person 1
2
person = new Person("Paul"); // ok -- stesso tipo 3
person = new Student("Sam", 123); // ok -- sottotipo 4
```

PRINCIPIO DI SOSTITUZIONE

Se S è una sottoclasse di T, allora gli oggetti T possono essere sostituiti da oggetti S

► Noto come Principio di Sostituzione di Liskov

```
Person person = new Person("Paul");           1
Student student = new Student("Sam", 123);      2
                                                3
person = student;    // ok -- sostituzione di sottotipo  4
student = person;    // errore -- sostituzione di supertipo  5
```

PRINCIPIO DI SOSTITUZIONE (CONT.)

Il compilatore effettua le chiamate ai metodi sulla base del tipo del riferimento

- ▶ Il riferimento `ref` è di tipo `Person`
- ▶ Vogliamo chiamare `ref.getName()`
- ▶ Il metodo `getName()` è definito nella classe `Person`?

Se il tipo del riferimento non fornisce il metodo, si ha un errore di compilazione

- ▶ Il subtyping garantisce che tale metodo esista

RAZIONALE DEL PRINCIPIO DI SOSTITUZIONE

Il tipo di un riferimento stabilisce i metodi che si possono chiamare su un oggetto

```
class Person {  
    String name;  
    Person(String name) {  
        this.name = name;  
    }  
    String getName() {  
        return name;  
    }  
}  
  
Person person = new Student("Sam", 123); // ok
```

```
class Student extends Person {  
    int matricola;  
    Student(String name, int matricola) {  
        super(name);  
        this.matricola = matricola;  
    }  
    int getMatricola() { return matricola; }  
}  
  
Student  
// violazione del principio di sostituzione
```

person.getName() 

► Metodo trovato (ereditato)

student.getMatricola() 

► Metodo non trovato

BINDING DINAMICO

E per quanto riguarda i metodi ridefiniti?

```
class Person extends Object {           1
    String name;                          2
    Person(String name) { this.name = name; } 3
    @Override // ridefinisce il metodo toString() della classe Object 4
    public Str                             5
}                                           6

Object obj = new Person("Sam");          7
                                           8
```

Quale metodo viene selezionato quando si chiama `obj.toString()`?

- ▶ Entrambe le classi `Object` e `Person` forniscono il metodo `toString()`

BINDING DINAMICO (CONT.)

Come vengono risolti i metodi dalla JVM

- ▶ La JVM recupera la classe di origine dell'oggetto target
 - ▶ Se la classe contiene il metodo da chiamare, questo viene eseguito
 - ▶ Altrimenti, viene considerata la classe genitore e il passo precedente viene ripetuto
 - ▶ È garantito che la procedura prima o poi termini
- 🔗 Il compilatore controlla che la gerarchia della classe del riferimento contenga il metodo

CONVERSIONE DI TIPO

Java è fortemente tipizzato, ogni variabile deve avere un tipo a tempo di compilazione

```
float f; 1
f = 4.7f; // o // ok -- valore float a variabile float 2
f = true; // e // errore -- valore boolean a variabile float 3
4
Car c; 5
c = new Car(); // ok -- oggetto Car a variabile Car 6
c = new String() // errore -- oggetto String a variabile Car 7
```


CONVERSIONE DI TIPO (CONT.)

Conversione di tipo: a volte è possibile cambiare il tipo di un'espressione

- In modo esplicito o implicito (da parte del compilatore)

```
int i; 1
float f; 2

f = i; / // cast implicito da int a float 3
4

f = (float) 42; / // cast esplicito da int a float 5
6
```

La conversione di tipo è anche detta **casting**

CASTING DI TIPI PRIMITIVI

Java esegue un cast implicito da tipi “più piccoli” a tipi “più grandi”

► Ma non viceversa

// dobbiamo forzare il casting

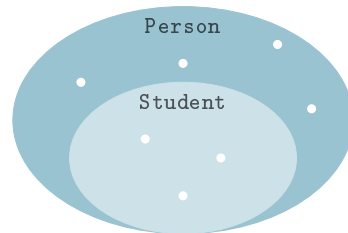
```
int i = 3; 1
float f = 2.4f; 2
3
f = i; // ok -- cast lecito (float più grande di int)
System.out.println(f); / // l'output sarà '3.0' 5
6
i = f; // err // errore -- cast non lecito (int più piccolo di float) 7
8
i = (int) f; // ok -- cast forzato (con perdita di informazione)
System.out.println(i); / // l'output sarà '2' 10
```

UPCASTING DI CLASSI

Upcast: da un tipo più specifico (sottotipo) a un tipo più generale (supertipo)

- ▶ L'upcasting viene eseguito implicitamente dal compilatore
- ▶ Il principio di sostituzione garantisce la type-safety

```
// Student estende Person           1
                                   2
Student student = new Student("Sam", 123); 3
                                   4
person = student; / // upcast implicito 5
```



$\forall \text{obj} \in \text{Student} . \text{obj} \in \text{Person}$

RAZIONALE DELL'UPCASTING DI CLASSI

Possiamo operare nello stesso modo su oggetti di classi diverse

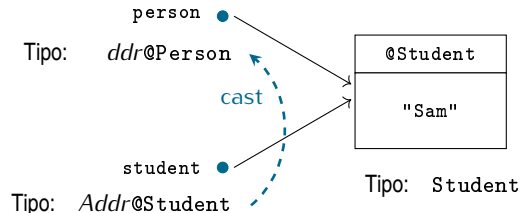
- A patto che ereditino da una classe comune

```
Person[] courseParticipants = {  
    new Lecturer("Paul", "Programming_II"),  
    new Student("Sam", 123),  
    new Student("Gary", 456)  
};  
  
for (Person person : courseParticipants)  
    System.out.println(person.toString());
```

1
2
3
4
5
6
7
8

È TUTTA UNA QUESTIONE DI RIFERIMENTI

Tipo di riferimento e tipo di oggetto sono concetti distinti



```
Person person = null;  
Student student = new Student("Sam", 123);  
person = student;
```

Il cast influenza solo il riferimento, l'oggetto puntato (valore) non cambia

- Solo con i tipi primitivi il casting comporta una conversione di valore

DOWNCASTING DI CLASSI

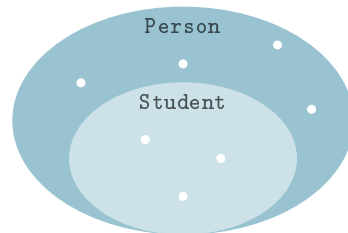
Downcast: da un tipo più generale (supertipo) a un tipo più specifico (sottotipo)

- ▶ Il downcasting deve essere forzato esplicitamente dal programmatore
- ▶ Non è garantito essere type-safe

// l'esempio sotto è sicuro?

```
// Student estende Person      1
Person person = new Person("Paul"); 2
Student student = new Student("Sam", 123); 3

student = (Student) person; // // downcast 5
                           // esplicito    6
```



$\exists \text{obj} \in \text{Person} . \text{obj} \notin \text{Student}$

RAZIONALE DEL DOWNCASTING DI CLASSI

Per accedere a un membro definito in una classe serve un riferimento di quel tipo

- ▶ O di una qualunque sottoclasse

```
Person person = courseParticipants[1];           1
System.out.println(person.getMatricola()); // errore del compilatore 2
// la classe Person non conosce il metodo getMatricola()
                                                    4
Student student = (Student) person;              5
System.out.println(student.getMatricola()); // ok per il compilatore
// la classe Student conosce il metodo getMatricola()
```

... purché `courseParticipants[1]` sia un oggetto `Student`

ATTENZIONE AI DOWNCAST

Il compilatore si fida di qualunque downcast (nella gerarchia di ereditarietà)

- ▶ Ma la JVM controlla a runtime la coerenza dei riferimenti
- ▶ Downcast scorretti sollevano un'eccezione

```
// Student estende Person 1
Person person = new Person("Paul"); 2
Student student = new Student("Sam", 123); 3
4

student = (Student) person; / // errore a runtime -- ClassCastException
```


ERRORI A TEMPO DI COMPILAZIONE E A RUNTIME

```
Person person = new Student("Sam", 123);
```

<code>person.getMatricola()</code>		errore a tempo di compilazione
<code>((Student) person).getMatricola()</code>		nessun errore
<code>((String) person).getMatricola()</code>		errore a tempo di compilazione
<code>((Lecturer) person).getCourse()</code>		errore a runtime

SICUREZZA DEL DOWNCAST

Per evitare errori a runtime, usare `ref instanceof Clazz`

- ▶ Per verificare se l'oggetto puntato da `ref` può essere convertito in `Clazz`
- ▶ Per verificare se appartiene a `Clazz` o a una qualunque sottoclasse

```
Person person = courseParticipants[1];           1
                                                    2
if (person instanceof Student) {                 3
    Student student = (Student) person;          4    // nessuna ClassCastException
    System.out.println(student.getMatric         5
}                                                  6
```

DOMANDE E RISPOSTE

